

Programme	:	Bachelor of Computer Science (Data Engineering)
Subject Code	:	SECJ 2154
Subject Name	:	OBJECT-ORIENTED PROGRAMMING
Session-Sem	:	2024/2025-2
MINI PROJECT TITLE	:	MyNoodle food ordering system (REPORT)

Prepared by

1.	WOO CHENG SHUAN	(A23CS0283)
:	2. ELIJAH SHE YU SHENG	(A23CS0073)
	3. MUHAMMAD MUKHRITZ AL	(A23CS0250)
	IMAN BIN MOHD RAFFI	

Section : 02

Lecturer : Dr. Zuraini Ali Shah

Date : 22-06-2025

TABLE OF CONTENT

No.	Section Title	Page
1.0	Introduction	3
2.0	Problem statement	3
3.0	Objective	4
4.0	Class Diagram of the project (PROJECT DESIGN)	4
	3.1 The Class Diagram Description	5
5.0	Project Output (Screenshots)	6-12
6.0	Conclusion	13
7.0	Appendices (java files)	13-24

1.0 Introduction

The MyNoodle Food Ordering System is designed to make it easier and faster to handle the whole process of ordering, preparing and delivering noodle dishes. It uses object-oriented programming to help manage things like creating and customizing the menu, handling customer profiles and their preferences, placing and tracking order in real time, processing payments safely, and organizing deliveries.

This system built to grow with the business and stay reliable even when handling many orders. It helps improve how the kitchen runs, keep track of stock and sale correctly and gives a smooth and user-friendly experience for both customer and kitchen staff.

2.0 Problem statement

Inefficient practices like long wait times, lost orders, inaccurate inventory tracking and inconsistent customer experiences can result from traditional noodle shops' frequent reliance on manual procedures for order taking, menu management and delivery tracking. Scaling operations and preserving service quality during peak hours are made more difficult by these constraints.

The MyNoodle Food Ordering System is proposed to address these challenges by introducing an automated, end-to-end digital solution to address these issues. Customizable menu administration, order placement, payment processing and tracking in real-time are some of the main features. By transitioning from manual to automated workflows, the system seeks to reduce errors, optimize kitchen and delivery operations, improve customer satisfaction and provide accurate inventory and sales insights.

3.0 Objective

The objective of the MyNoodle Food Ordering System is to develop an efficient and user-friendly digital platform that streamlines the entire noodle ordering and delivery process. This system aims to automate key operations including menu customization, customer profile management, real-time order placement, payment processing, and delivery tracking. By transitioning from manual to automated workflows, the system reduces human errors, enhances kitchen and delivery efficiency, and provides customers with a consistent and satisfying experience. Additionally, the system is designed to support inventory management and sales tracking, allowing the business to scale operations while maintaining service quality. The overall goal is to improve operational accuracy, save time, boost customer satisfaction, and ensure the business can effectively handle high volumes of orders, particularly during peak periods.

4.0 Class Diagram of the project

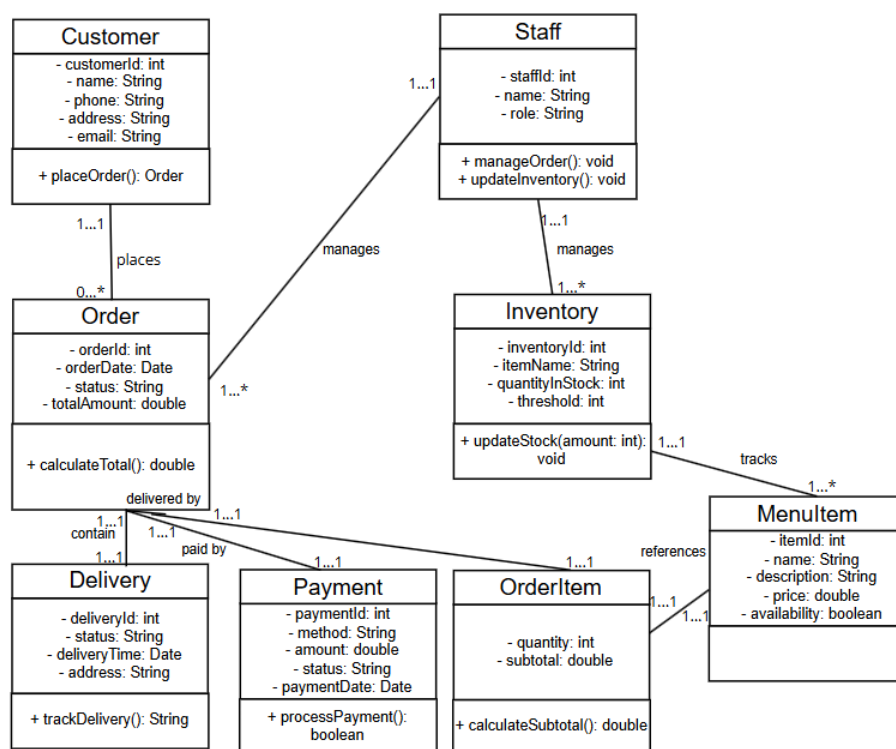


Table 3.0.1 MyNoodle Food Ordering System Class Diagram

The MyNoodle Food Ordering System is designed using object-oriented principles with interconnected classes to manage the full ordering workflow. The Customer class allows users to place orders, which are handled by the Order class containing multiple OrderItem instances. Each OrderItem is linked to a MenuItem representing dishes on the menu. The Order is associated with both a Payment for transaction details and a Delivery to track the delivery status. Inventory control is

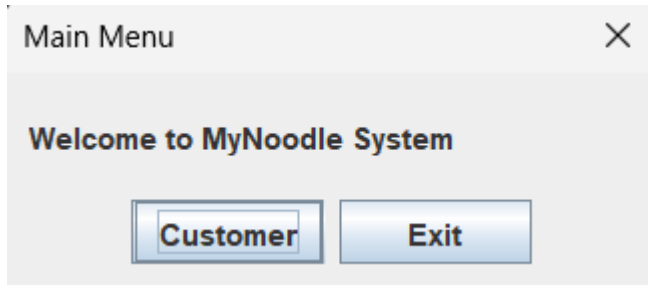
managed through the Inventory class, which monitors stock levels of menu items. The Staff class oversees both order and inventory management, ensuring smooth kitchen and delivery operations. All classes are interlinked to support real-time ordering, payment, and delivery in a seamless and automated manner.

4.1 The Class Diagram Description

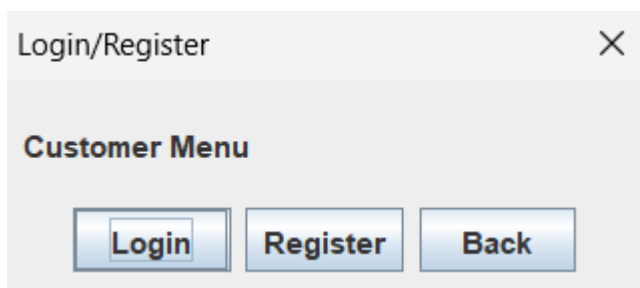
Class Name	Description	Key Attributes	Relationships
Customer	Represents a user who places food orders.	customerId, name, phone, address, email	- Places one or more Order
MenuItem	Represents an individual item on the menu.	itemId, name, description, price, availability	- Referenced by OrderItem- Tracked by Inventory
Order	Represents a customer's complete order.	orderId, orderDate, status, totalAmount	- Placed by Customer- Contains OrderItem- Linked to one Payment and one Delivery- Managed by Staff
OrderItem	Represents each individual item within an order.	quantity, subtotal	- References one MenuItem- Part of one Order
Payment	Records payment information for an order.	paymentId, method, amount, status, paymentDate	- Linked to one Order
Delivery	Tracks the delivery status of an order.	deliveryId, status, deliveryTime, address	- Linked to one Order
Inventory	Tracks menu ingredients or item stock levels.	inventoryId, itemName, quantityInStock, threshold	- Tracks multiple MenuItem- Managed by Staff
Staff	Represents an admin or kitchen/delivery worker.	staffId, name, role	- Manages multiple Order and Inventory

5.0 Project Output (Screenshots)

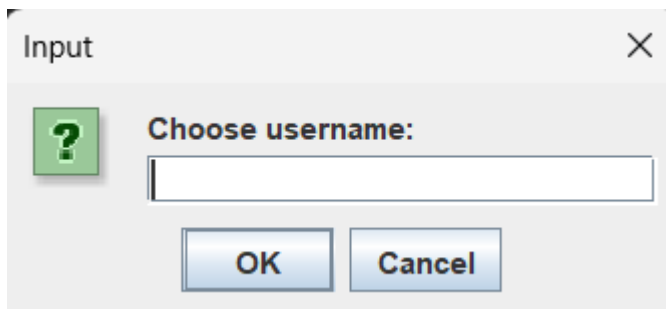
Register



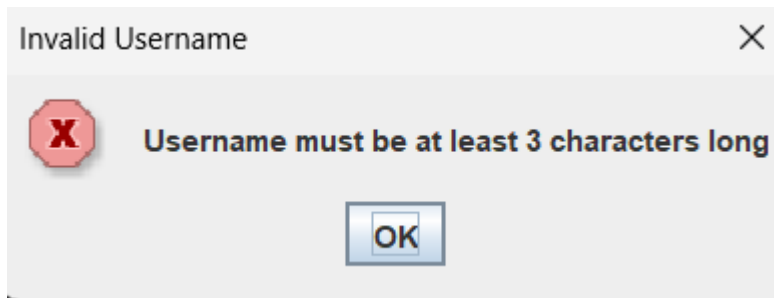
- Screen showing the main menu of the system where it welcomes the user to the system.



- Screen showing the login or register page where the user needs to register before they can use the system.



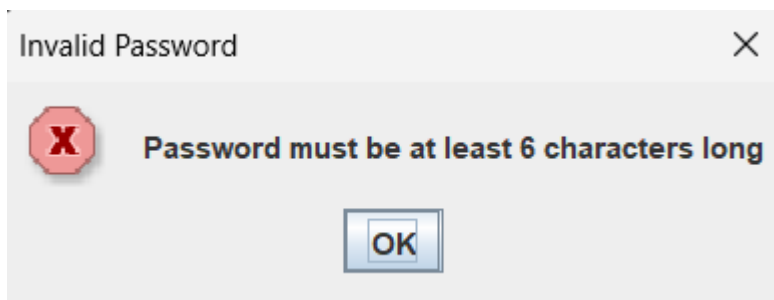
- Screen showing a form that needed the user to input their username to be registered.



- Screen showing the registration validation error as the user input username less than 3 characters long

A dialog box titled "Input" with a close button (X) in the top right corner. It features a green square icon with a white question mark on the left. The text "Choose password:" is displayed in the center. Below the text is a text input field. At the bottom, there are two buttons: "OK" and "Cancel".

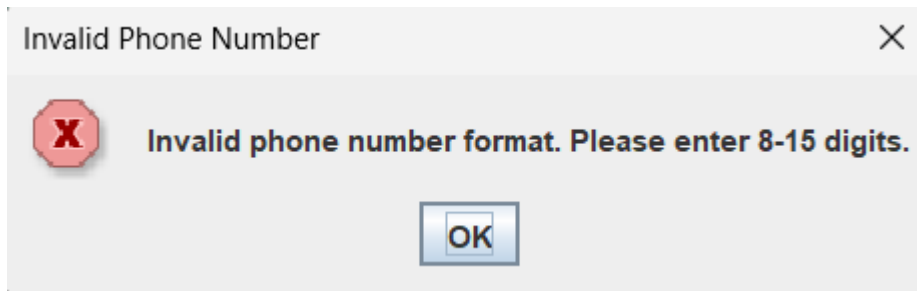
- Screen showing a form that needed the user to input their username to be registered.



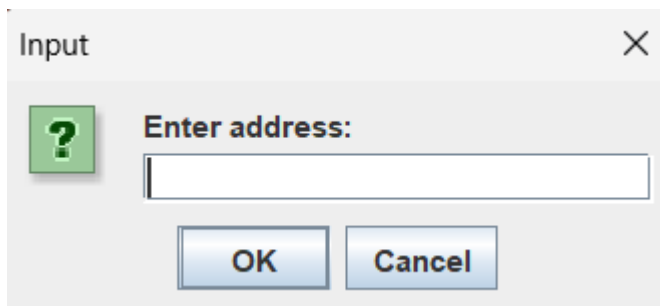
- Screen showing the registration validation error as the user input username less than 3 characters long

A dialog box titled "Input" with a close button (X) in the top right corner. It features a green square icon with a white question mark on the left. The text "Enter phone number:" is displayed in the center. Below the text is a text input field. At the bottom, there are two buttons: "OK" and "Cancel".

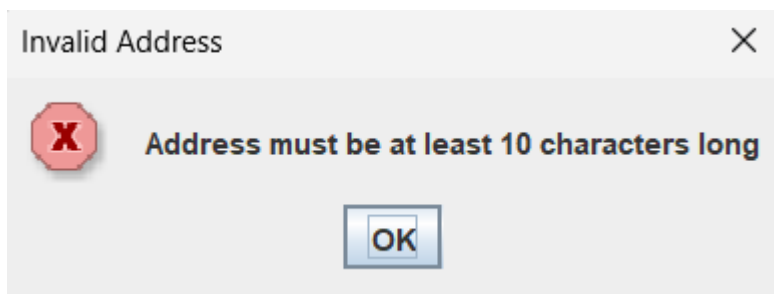
- Screen showing a form that needed the user to input their phone number to be registered.



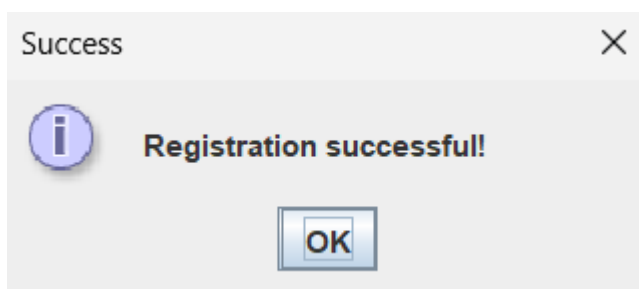
- Screen showing the registration validation error as the user input username less than 3 characters long



- Screen showing a form that needed the user to input their phone number to be registered.

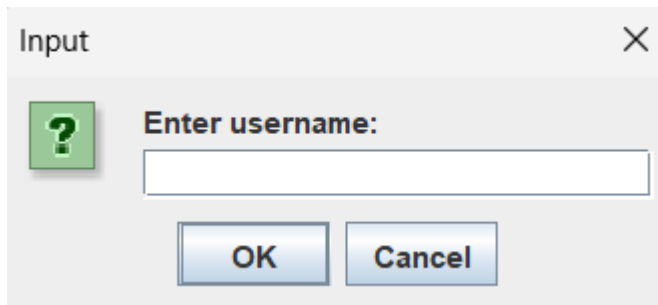


- Screen showing the registration validation error as the user input username less than 3 characters long

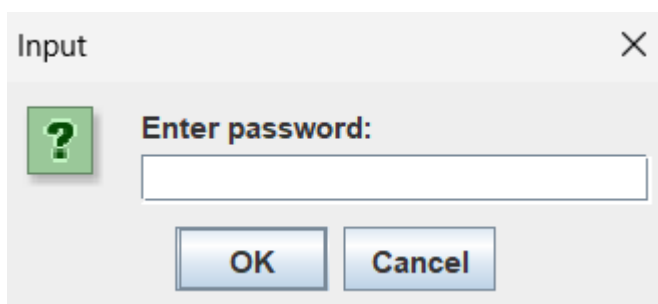


- A confirmation message ("Registration successful!") with a button to proceed to the Login page.

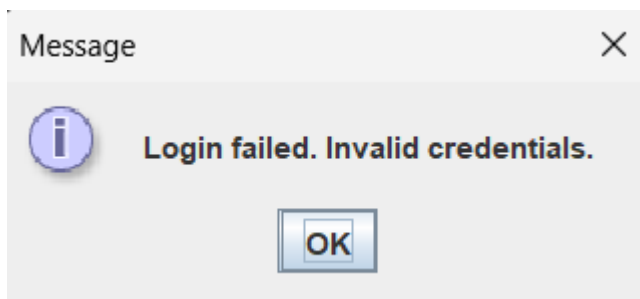
Login

A screenshot of a Windows-style 'Input' dialog box. It has a title bar with the word 'Input' and a close button (X). Inside, there is a green square icon with a white question mark. To the right of the icon is the text 'Enter username:'. Below this text is a single-line text input field. At the bottom of the dialog are two buttons: 'OK' and 'Cancel'.

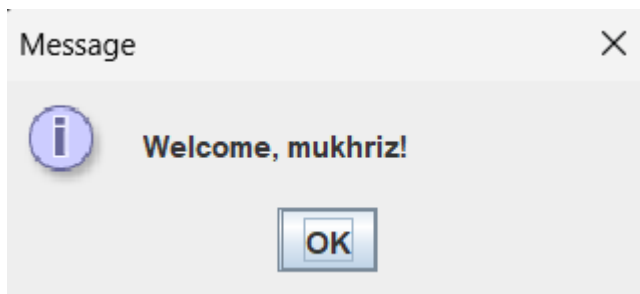
- Screen showing a form that needed the user to enter their username to login to their account.

A screenshot of a Windows-style 'Input' dialog box. It has a title bar with the word 'Input' and a close button (X). Inside, there is a green square icon with a white question mark. To the right of the icon is the text 'Enter password:'. Below this text is a single-line text input field. At the bottom of the dialog are two buttons: 'OK' and 'Cancel'.

- Screen showing a form that needed the user to enter their password to login to their account.

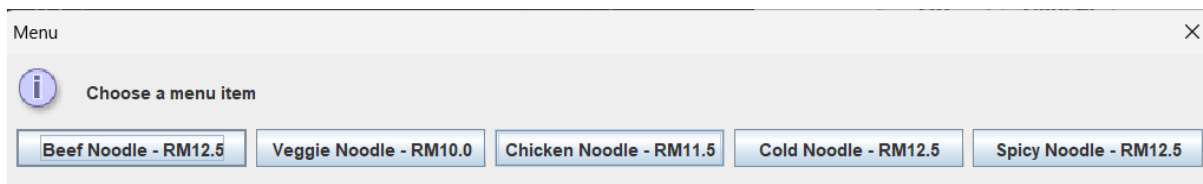
A screenshot of a Windows-style 'Message' dialog box. It has a title bar with the word 'Message' and a close button (X). Inside, there is a blue circular icon with a white 'i' (information). To the right of the icon is the text 'Login failed. Invalid credentials.'. Below this text is a single 'OK' button.

- An alert indicating the email or password combination is invalid.

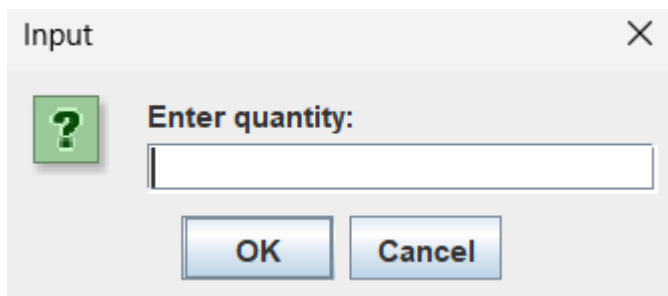
A screenshot of a Windows-style 'Message' dialog box. It has a title bar with the word 'Message' and a close button (X). Inside, there is a blue circular icon with a white 'i' (information). To the right of the icon is the text 'Welcome, mukhriz!'. Below this text is a single 'OK' button.

- A confirmation message ("Welcome, user") with a button to proceed to the ordering page.

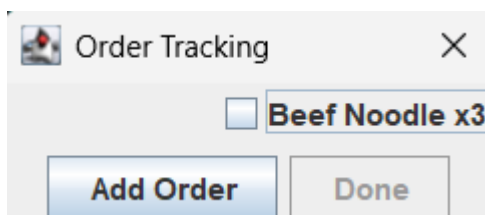
Order



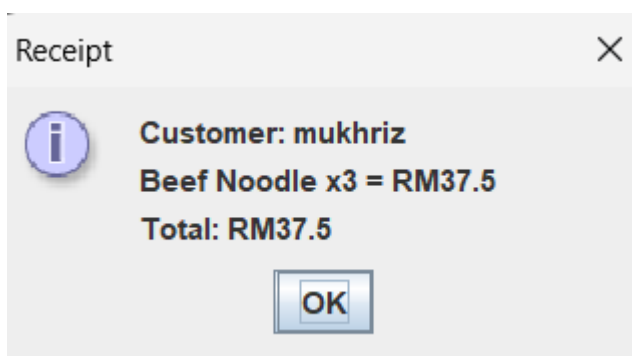
- Screen showing the list of a menu item the user can choose to order.



- Screen showing a form that needed the user to enter the quantity of the item chosen.



- Screen showing the order tracking where the user can add other menu or they can complete their order



- Screen showing the receipt order for the customer

```

≡ customers.txt
1  naimdaniel123,123456,naim,012345678,naim@gmail.com,naim@gmail.com
2  naim,1234567,naim,01234567,utm malaysia,naim@gmail.com
3  naimdaniel,12345678,naim,012345678,utm malaysia,naim@gmail.com
4  mukhriz,123456,mukhriz,012345678,utm malaysia,mukhriz@gmail.com
5  afiq,123456,afiq,012345678,utm malaysia,afiq@gmail.com

```

- Screen showing the list of customers registered to the system.

```

≡ daily_sales_report_20250620.txt X
≡ daily_sales_report_20250620.txt
1  =====
2  Daily Sales Summary
3  =====
4  Date: 2025-06-20
5  Total Orders: 1
6  Total Sales: RM37.50
7  -----
8  Item                               Total Qty
9  -----
10 Beef Noodle                        3
11  =====
12

```

- Screen showing the daily sales summary of the food ordering system.

```
receipt.txt X
receipt.txt
1  =====
2  |      |      |      |      |      |      |      |      |      |      |
3  |      |      |      |      |      |      |      |      |      |      |
4  Order ID: 002
5  Date: 2025-06-20 10:28:51
6  Customer: afiq
7  Phone: 012345678
8  Address: utm malaysia
9  -----
10 Item                      Qty    Subtotal
11 -----
12 Beef Noodle                2      25.00
13 -----
14 Total:                      RM25.00
15 =====
16 Thank you for dining with us!
17
```

- Screen showing receipt for each customer after finish ordering

```
sales_log.txt
1  2025-06-20 | 001 | mukhriz | 37.50 | Beef Noodle x3
```

- Screen showing the sales log for the restaurant.

6.0 Conclusion

The MyNoodle Food Ordering System delivers a comprehensive, object-oriented solution that automates every step of noodle shop operations—from dynamic menu customization and real-time order placement to secure payments, delivery tracking, and inventory management. Its modular class design (Customer, Order, MenuItem, Payment, Delivery, Inventory, etc.) ensures clear separation of concerns, data consistency, and maintainability, resulting in fewer order errors, faster service, and accurate stock and sales reporting.

Looking ahead, MyNoodle can evolve with integrated analytics dashboards for sales trends, customer loyalty programs to boost retention, and multi-restaurant support for scaling. These enhancements will further streamline kitchen workflows, empower staff coordination during peak times, and elevate the overall customer experience, solidifying MyNoodle as a robust foundation for digital transformation in the food service industry

7.0 Appendices (java files)

Customer.java

```
1 public class Customer extends User {
2     private String username, password, phone, address, email;
3
4     public Customer(String id, String username, String password, String name, String phone, String address, String email) {
5         super(id, name);
6         this.username = username;
7         this.password = password;
8         this.phone = phone;
9         this.address = address;
10        this.email = email;
11    }
12
13    public String getUsername() { return username; }
14    public String getPassword() { return password; }
15    public String getPhone() { return phone; }
16    public String getAddress() { return address; }
17    public String getEmail() { return email; }
18 };
```

CustomerDatabase.java

```
1 import java.io.*;
2 import java.util.*;    The import java.util is never used
3
4 public class CustomerDatabase {
5     private static final String FILE_NAME = "customers.txt";
6
7     Tabnine | Edit | Test | Explain | Document
8     public static boolean registerCustomer(Customer customer) {
9         if (isUsernameTaken(customer.getUsername())) return false;
10        try (PrintWriter writer = new PrintWriter(new FileWriter(FILE_NAME, append:true))) {
11            writer.println(customer.getUsername() + "," + customer.getPassword() + "," + customer.getName() + "," +
12                customer.getPhone() + "," + customer.getAddress() + "," + customer.getEmail());
13        } catch (IOException e) {
14            return false;
15        }
16    }
17
18     Tabnine | Edit | Test | Explain | Document
19     public static Customer login(String username, String password) {
20        try (BufferedReader reader = new BufferedReader(new FileReader(FILE_NAME))) {
21            String line;
22            while ((line = reader.readLine()) != null) {
23                String[] data = line.split(regex:",");
24                if (data.length >= 6 && data[0].equals(username) && data[1].equals(password)) {
25                    return new Customer(id:"C001", data[0], data[1], data[2], data[3], data[4], data[5]);
26                }
27            } catch (IOException e) {
28                e.printStackTrace();
29            }
30            return null;
31        }
32
33     Tabnine | Edit | Test | Explain | Document
34     private static boolean isUsernameTaken(String username) {
35        try (BufferedReader reader = new BufferedReader(new FileReader(FILE_NAME))) {
36            String line;
37            while ((line = reader.readLine()) != null) {
38                String[] data = line.split(regex:",");
39                if (data.length >= 1 && data[0].equals(username)) return true;
40            } catch (IOException e) {
41                // File might not exist yet, which is okay
42            }
43            return false;
44        }
45    }
```

Main.java

```
1  import javax.swing.*;
2
3  public class Main {
4
5      Tabnine | Edit | Test | Explain | Document
6      private static String generateOrderId() {
7          int nextCounter = getNextOrderCounter();
8          return String.format(format:"%03d", nextCounter);
9      }
10
11      Tabnine | Edit | Test | Explain | Document
12      private static int getNextOrderCounter() {
13          int maxCounter = 0;
14          try {
15              java.nio.file.Path salesLogPath = java.nio.file.Paths.get(first:"sales_log.txt");
16              if (java.nio.file.Files.exists(salesLogPath)) {
17                  java.util.List<String> lines = java.nio.file.Files.readAllLines(salesLogPath);
18                  for (String line : lines) {
19                      String[] parts = line.split(regex:"\\|");
20                      if (parts.length >= 2) {
21                          String orderId = parts[1].trim();
22                          try {
23                              // Try to parse the order ID as a simple number
24                              int counter = Integer.parseInt(orderId);
25                              if (counter > maxCounter) {
26                                  maxCounter = counter;
27                              }
28                          } catch (NumberFormatException e) {
29                              // Skip invalid counter formats
30                          }
31                      }
32                  }
33              } catch (Exception e) {
34                  System.out.println("Error reading sales log: " + e.getMessage());
35              }
36          }
37          return maxCounter + 1;
38      }
39
40      Run | Debug | Tabnine | Edit | Test | Explain | Document
41      public static void main(String[] args) {
42          Customer customer = null;
43          while (customer == null) {
44              int choice = JOptionPane.showOptionDialog(parentComponent:null, message:"Welcome to MyNoodle System", title:"Main Menu",
45                  JOptionPane.DEFAULT_OPTION, JOptionPane.PLAIN_MESSAGE, icon:null,
46                  new String[]{"Customer", "Exit"}, initialValue:"Customer");
47
48              if (choice == 0) {
49                  customer = handleLogin();
50              } else {
51                  System.exit(status:0);
52              }
53          }
54          runOrderingFlow(customer, order:null);
55      }
56
57      private static Customer handleLogin() {
58          int choice = JOptionPane.showOptionDialog(parentComponent:null, message:"Customer Menu", title:"Login/Register",
59              JOptionPane.DEFAULT_OPTION, JOptionPane.PLAIN_MESSAGE, icon:null,
60              new String[]{"Login", "Register", "Back"}, initialValue:"Login");
61
62          if (choice == 0) {
63              return loginCustomer();
64          } else if (choice == 1) {
65              registerCustomer();
66          }
67          return null;
68      }
69
70      Tabnine | Edit | Test | Explain | Document
71      private static String getValidatedInput(String prompt, String fieldName, java.util.function.Consumer<String> validator) {
72          while (true) {
73              String input = JOptionPane.showInputDialog(prompt);
74              if (input == null) return null; // User cancelled
75              try {
76                  validator.accept(input);
77                  return input;
78              } catch (IllegalArgumentException e) {
79                  JOptionPane.showMessageDialog(parentComponent:null, e.getMessage(), "Invalid " + fieldName, JOptionPane.ERROR_MESSAGE);
80              }
81          }
82      }
83  }
```

```

80 private static void registerCustomer() {
81     try {
82         String username = getValidatedInput(prompt:"Choose username:", fieldName:"Username", ValidationUtils::validateUsername);
83         if (username == null) return;
84
85         String password = getValidatedInput(prompt:"Choose password:", fieldName:"Password", ValidationUtils::validatePassword);
86         if (password == null) return;
87
88         String name = getValidatedInput(prompt:"Enter name:", fieldName:"Name", ValidationUtils::validateName);
89         if (name == null) return;
90
91         String phone = getValidatedInput(prompt:"Enter phone number:", fieldName:"Phone Number", ValidationUtils::validatePhone);
92         if (phone == null) return;
93
94         String address = getValidatedInput(prompt:"Enter address:", fieldName:"Address", ValidationUtils::validateAddress);
95         if (address == null) return;
96
97         String email = getValidatedInput(prompt:"Enter email:", fieldName:"Email", ValidationUtils::validateEmail);
98         if (email == null) return;
99
100        Customer newCustomer = new Customer(id:"C001", username, password, name, phone, address, email);
101        if (CustomerDatabase.registerCustomer(newCustomer)) {
102            JOptionPane.showMessageDialog(parentComponent:null, message:"Registration successful!", title:"Success", JOptionPane.INFORMATION_MESSAGE);
103        } else {
104            JOptionPane.showMessageDialog(parentComponent:null, message:"Username already taken!", title:"Registration Failed", JOptionPane.ERROR_MESSAGE);
105        }
106    } catch (Exception e) {
107        JOptionPane.showMessageDialog(parentComponent:null, "Unexpected error during registration: " + e.getMessage(),
108            title:"Registration Error", JOptionPane.ERROR_MESSAGE);
109    }
110 }
111
112 Tabnine | Edit | Test | Explain | Document
113 private static Customer loginCustomer() {
114     try {
115         String username = JOptionPane.showInputDialog(message:"Enter username:");
116         String password = JOptionPane.showInputDialog(message:"Enter password:");
117         Customer customer = CustomerDatabase.login(username, password);
118         if (customer != null) {
119             JOptionPane.showMessageDialog(parentComponent:null, "Welcome, " + customer.getName() + "!");
120             return customer;
121         } else {
122             JOptionPane.showMessageDialog(parentComponent:null, message:"Login failed. Invalid credentials.");
123         }
124     } catch (Exception e) {
125         JOptionPane.showMessageDialog(parentComponent:null, "Login error: " + e.getMessage());
126     }
127     return null;
128 }
129
130 Tabnine | Edit | Test | Explain | Document
131 private static void runOrderingFlow(Customer customer, Order order) {
132     MenuItem[] menuItems = {
133         new MenuItem(itemId:"M001", name:"Beef Noodle", description:"Spicy beef soup", price:12.50, availability:true),
134         new MenuItem(itemId:"M002", name:"Veggie Noodle", description:"Vegetarian", price:10.00, availability:true),
135         new MenuItem(itemId:"M003", name:"Chicken Noodle", description:"Chicken soup", price:11.50, availability:true),
136         new MenuItem(itemId:"M004", name:"Cold Noodle", description:"Cold noodle dish", price:12.50, availability:true),
137         new MenuItem(itemId:"M005", name:"Spicy Noodle", description:"Extra spicy soup", price:12.50, availability:true)
138     };
139
140     String[] options = java.util.Arrays.stream(menuItems)
141         .map(item -> item.getName() + " - RM" + item.getPrice())
142         .toArray(String[]::new);
143
144     int choice = JOptionPane.showOptionDialog(parentComponent:null, message:"Choose a menu item", title:"Menu",
145         JOptionPane.DEFAULT_OPTION, JOptionPane.INFORMATION_MESSAGE, icon:null, options, options[0]);
146
147     if (choice == JOptionPane.CLOSED_OPTION || choice < 0 || choice >= menuItems.length) return;
148
149     try {
150         int quantity = Integer.parseInt(JOptionPane.showInputDialog(message:"Enter quantity:"));
151         if (quantity == null) {
152             order = new Order(generateOrderId(), customer);
153         }
154         order.addItem(new OrderItem(menuItems[choice], quantity));
155         order.saveReceiptOnly();
156         showOrderTrackingScreen(order, customer);
157     } catch (NumberFormatException e) {
158         JOptionPane.showMessageDialog(parentComponent:null, message:"Please enter a valid number for quantity.", title:"Invalid Input", JOptionPane.ERROR_MESSAGE);
159     }
160 }

```



```

160 private static void showOrderTrackingScreen(Order order, Customer customer) {
161     java.util.List<OrderItem> items = order.getItems();
162     JPanel panel = new JPanel();
163     panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
164
165     JCheckBox[] checkBoxes = items.stream()
166         .map(item -> new JCheckBox(item.getMenuItem().getName() + " x" + item.getQuantity()))
167         .toArray(JCheckBox[]::new);
168
169     java.util.Arrays.stream(checkBoxes).forEach(panel::add);
170
171     JButton addOrderBtn = new JButton(text:"Add Order");
172     JButton doneBtn = new JButton(text:"Done");
173     doneBtn.setEnabled(b:false);
174
175     JPanel btnPanel = new JPanel();
176     btnPanel.add(addOrderBtn);
177     btnPanel.add(doneBtn);
178     panel.add(btnPanel);
179
180     java.awt.event.ItemListener checkListener = e ->
181         doneBtn.setEnabled(java.util.Arrays.stream(checkBoxes).allMatch(JCheckBox::isSelected));
182
183     java.util.Arrays.stream(checkBoxes).forEach(cb -> cb.addItemListener(checkListener));
184
185     JDialog dialog = new JDialog();
186     dialog.setTitle(title:"Order Tracking");
187     dialog.setModal(modal:true);
188     dialog.setContentPane(panel);
189     dialog.pack();
190     dialog.setLocationRelativeTo(c:null);
191
192     addOrderBtn.addActionListener(e -> {
193         dialog.dispose();
194         runOrderingFlow(customer, order);
195     });
196     doneBtn.addActionListener(e -> {
197         dialog.dispose();
198         order.finalizeOrder();
199         showReceiptDialog(order, customer);
200     });
201
202     dialog.setVisible(b:true);
203 }
204
205 private static void showReceiptDialog(Order order, Customer customer) {
206     StringBuilder sb = new StringBuilder();
207     sb.append(str:"Customer: ").append(customer.getName()).append(str:"\n");
208
209     order.getItems().forEach(item ->
210         sb.append(item.getMenuItem().getName())
211             .append(str:" x").append(item.getQuantity())
212             .append(str:" = RM").append(item.getSubtotal()).append(str:"\n"));
213
214     sb.append(str:"Total: RM").append(order.getTotalAmount());
215     JOptionPane.showMessageDialog(parentComponent:null, sb.toString(), title:"Receipt", JOptionPane.INFORMATION_MESSAGE);
216     main(args:null);
217 }
218 }

```

Tabnine | Edit | Test | Explain | Document

MenuItem.java

```
1  ✓ public class MenuItem {  
2      private String itemId, name, description;    The value of the field MenuItem.description is not used  
3      private double price;  
4      private boolean availability;    The value of the field MenuItem.availability is not used  
5  
6  ✓  public MenuItem(String itemId, String name, String description, double price, boolean availability) {  
7      this.itemId = itemId;  
8      this.name = name;  
9      this.description = description;  
10     this.price = price;  
11     this.availability = availability;  
12 }  
13  
14     Tabnine | Edit | Test | Explain | Document  
15     public String getItemId() { return itemId; }  
16     Tabnine | Edit | Test | Explain | Document  
17     public String getName() { return name; }  
18     Tabnine | Edit | Test | Explain | Document  
19     public double getPrice() { return price; }  
20 }
```

Order.java

```

1  import java.io.*;
2  import java.nio.file.*;
3  import java.text.SimpleDateFormat;
4  import java.util.*;
5
6  public class Order {
7      private String orderId;
8      private Customer customer;
9      private List<OrderItem> items = new ArrayList<>();
10     private static final SimpleDateFormat DATE_FORMAT = new SimpleDateFormat(pattern:"yyyy-MM-dd");
11     private static final SimpleDateFormat DATETIME_FORMAT = new SimpleDateFormat(pattern:"yyyy-MM-dd HH:mm:ss");
12
13     public Order(String orderId, Customer customer) {
14         this.orderId = orderId;
15         this.customer = customer;
16     }
17
18     Tabnine | Edit | Test | Explain | Document
19     public void addItem(OrderItem item) {
20         items.add(item);
21     }
22
23     Tabnine | Edit | Test | Explain | Document
24     public double getTotalAmount() {
25         return items.stream().mapToDouble(OrderItem::getSubtotal).sum();
26     }
27
28     Tabnine | Edit | Test | Explain | Document
29     // Method to save receipt only (without logging sales)
30     public void saveReceiptOnly() {
31         saveReceipt();
32     }
33
34     Tabnine | Edit | Test | Explain | Document
35     // Method to finalize order and log sales
36     public void finalizeOrder() {
37         saveReceipt();
38         logSales();
39         updateDailyReport();
40     }
41
42     Tabnine | Edit | Test | Explain | Document
43     public void saveToFile() {
44         saveReceipt();
45         logSales();
46         updateDailyReport();
47     }
48
49     Tabnine | Edit | Test | Explain | Document
50     private void saveReceipt() {
51         try (PrintWriter writer = new PrintWriter(new FileWriter(fileName:"receipt.txt"))) {
52             writer.println(x:"-----");
53             writer.println(x:"          MyNoodle Receipt          ");
54             writer.println(x:"-----");
55             writer.println("Order ID: " + orderId);
56             writer.println("Date: " + DATETIME_FORMAT.format(new Date()));
57             writer.println("Customer: " + customer.getName());
58             writer.println("Phone: " + customer.getPhone());
59             writer.println("Address: " + customer.getAddress());
60             writer.println(x:"-----");
61             writer.printf(format:"%-20s %5s %10s\n", ...args:"Item", "Qty", "Subtotal");
62             writer.println(x:"-----");
63             for (OrderItem item : items) {
64                 writer.printf(format:"%-20s %5d %10.2f\n",
65                     item.getMenuItem().getName(), item.getQuantity(), item.getSubtotal());
66             }
67             writer.println(x:"-----");
68             writer.printf(format:"%-26s RM%.2f\n", ...args:"Total:", getTotalAmount());
69             writer.println(x:"-----");
70             writer.println(x:"Thank you for dining with us!");
71         } catch (IOException e) {
72             System.out.println("Error saving receipt: " + e.getMessage());
73         }
74     }
75 }

```

```

70 private void logSales() {
71     try (PrintWriter writer = new PrintWriter(new FileWriter(fileName:"sales_log.txt", append:true))) {
72         String itemsStr = items.stream()
73             .map(item -> item.getMenuItem().getName() + " x" + item.getQuantity())
74             .reduce((a, b) -> a + ", " + b)
75             .orElse(other:"");
76
77         writer.printf(format:"%s | %s | %s | %.2f | %s\n",
78             DATE_FORMAT.format(new Date()), orderId, customer.getName(), getTotalAmount(), itemsStr);
79     } catch (IOException e) {
80         System.out.println("Error logging sales: " + e.getMessage());
81     }
82 }
83
84 Tabnine | Edit | Test | Explain | Document
85 private void updateDailyReport() {
86     try {
87         String today = DATE_FORMAT.format(new Date());
88         String reportFile = "daily_sales_report_" + today.replace(target:"-", replacement:"") + ".txt";
89
90         Map<String, Integer> itemQtyMap = new HashMap<>();
91         double totalSales = 0.0;
92         int totalOrders = 0;
93
94         if (Files.exists(Paths.get(first:"sales_log.txt"))) {
95             List<String> lines = Files.readAllLines(Paths.get(first:"sales_log.txt"));
96             for (String line : lines) {
97                 if (line.startsWith(today)) {
98                     String[] parts = line.split(regex:"\\|");
99                     if (parts.length >= 5) {
100                         totalSales += Double.parseDouble(parts[3].trim());
101                         totalOrders++;
102                         parseItems(parts[4].trim(), itemQtyMap);
103                     }
104                 }
105             }
106
107             writeDailyReport(reportFile, today, totalOrders, totalSales, itemQtyMap);
108         } catch (Exception e) {
109             System.out.println("Error updating daily report: " + e.getMessage());
110         }
111     }
112
113 Tabnine | Edit | Test | Explain | Document
114 private void parseItems(String itemsPart, Map<String, Integer> itemQtyMap) {
115     for (String entry : itemsPart.split(regex:" ", "")) {
116         int xIdx = entry.lastIndexOf(str:" x");
117         Click to add a breakpoint (xIdx > 0) {
118             String itemName = entry.substring(beginIndex:0, xIdx).trim();
119             int qty = Integer.parseInt(entry.substring(xIdx + 2).trim());
120             itemQtyMap.merge(itemName, qty, Integer::sum);
121         }
122     }
123
124 Tabnine | Edit | Test | Explain | Document
125 private void writeDailyReport(String reportFile, String today, int totalOrders, double totalSales, Map<String, Integer> itemQtyMap) throws IOException {
126     try (PrintWriter writer = new PrintWriter(new FileWriter(reportFile))) {
127         writer.println(x:"-----");
128         writer.println(x:"    Daily Sales Summary    ");
129         writer.println(x:"-----");
130         writer.println("Date: " + today);
131         writer.println("Total Orders: " + totalOrders);
132         writer.printf(format:"Total Sales: RM%.2f\n", totalSales);
133         writer.println(x:"-----");
134         writer.printf(format:"%-20s %10s\n", "Item", "Total Qty");
135         writer.println(x:"-----");
136         itemQtyMap.forEach((item, qty) -> writer.printf(format:"%-20s %10d\n", item, qty));
137         writer.println(x:"-----");
138     }
139
140 Tabnine | Edit | Test | Explain | Document
141 public List<OrderItem> getItems() {
142     return items;
143 }

```

OrderItem.java

```
1  public class OrderItem {
2      private MenuItem menuItem;
3      private int quantity;
4
5      public OrderItem(MenuItem menuItem, int quantity) {
6          this.menuItem = menuItem;
7          this.quantity = quantity;
8      }
9
10     Tabnine | Edit | Test | Explain | Document
11     public double getSubtotal() {
12         return menuItem.getPrice() * quantity;
13     }
14
15     Tabnine | Edit | Test | Explain | Document
16     public MenuItem getMenu() { return menuItem; }
17     Tabnine | Edit | Test | Explain | Document
18     public int getQuantity() { return quantity; }
19 }
```

Payable.java

```
1  public interface Payable {
2      void processPayment();
3  }
4
```

Payment.java

```
1  public class Payment implements Payable {
2      private String method;
3      private double amount;
4
5      public Payment(String method, double amount) {
6          this.method = method;
7          this.amount = amount;
8      }
9
10     Tabnine | Edit | Test | Explain | Document
11     @Override
12     public void processPayment() {
13         System.out.println("Payment of RM" + amount + " using " + method + " is processed.");
14     }
15 }
```

User.java

```
1  public abstract class User {
2      private String id, name;
3
4      public User(String id, String name) {
5          this.id = id;
6          this.name = name;
7      }
8
9      public String getId() { return id; }
10     public String getName() { return name; }
11 }
```

ValidationUtils.java

```
1  import java.util.regex.Pattern;
2
3  public class ValidationUtils {
4
5      // Email validation pattern
6      private static final Pattern EMAIL_PATTERN = Pattern.compile(
7          regex: "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$"
8      );
9
10     // Phone number validation pattern (supports various formats)
11     private static final Pattern PHONE_PATTERN = Pattern.compile(
12         regex: "[+]?[0-9]{8,15}$"
13     );
14
15     /**
16      * Validates email address format
17      * @param email the email to validate
18      * @throws IllegalArgumentException if email is invalid
19      */
20     public static void validateEmail(String email) throws IllegalArgumentException {
21         if (email == null || email.trim().isEmpty()) {
22             throw new IllegalArgumentException(s:"Email cannot be empty");
23         }
24
25         if (!EMAIL_PATTERN.matcher(email.trim()).matches()) {
26             throw new IllegalArgumentException(s:"Invalid email format. Please enter a valid email address.");
27         }
28     }
29
30     /**
31      * Validates phone number format
32      * @param phone the phone number to validate
33      * @throws IllegalArgumentException if phone number is invalid
34      */
35     public static void validatePhone(String phone) throws IllegalArgumentException {
36         if (phone == null || phone.trim().isEmpty()) {
37             throw new IllegalArgumentException(s:"Phone number cannot be empty");
38         }
39
40         // Remove spaces, dashes, and parentheses for validation
41         String cleanPhone = phone.replaceAll(regex:"[\\s\\-\\(\\)]", replacement:"");
42
43         if (!PHONE_PATTERN.matcher(cleanPhone).matches()) {
44             throw new IllegalArgumentException(s:"Invalid phone number format. Please enter 8-15 digits.");
45         }
46     }
47
48     /**
49      * Validates address format
50      * @param address the address to validate
51      * @throws IllegalArgumentException if address is invalid
52      */
53     public static void validateAddress(String address) throws IllegalArgumentException {
54         if (address == null || address.trim().isEmpty()) {
55             throw new IllegalArgumentException(s:"Address cannot be empty");
56         }
57
58         if (address.trim().length() < 10) {
59             throw new IllegalArgumentException(s:"Address must be at least 10 characters long");
60         }
61     }
62 }
```

```

62     if (address.trim().length() > 200) {
63         throw new IllegalArgumentException(s:"Address cannot exceed 200 characters");
64     }
65 }
66
67 Tabnine | Edit | Test | Explain | Document
68 /**
69  * Validates username format
70  * @param username the username to validate
71  * @throws IllegalArgumentException if username is invalid
72  */
73 public static void validateUsername(String username) throws IllegalArgumentException {
74     if (username == null || username.trim().isEmpty()) {
75         throw new IllegalArgumentException(s:"Username cannot be empty");
76     }
77
78     if (username.trim().length() < 3) {
79         throw new IllegalArgumentException(s:"Username must be at least 3 characters long");
80     }
81
82     if (username.trim().length() > 20) {
83         throw new IllegalArgumentException(s:"Username cannot exceed 20 characters");
84     }
85
86     if (!username.matches(regex:"^[a-zA-Z0-9_]+$")) {
87         throw new IllegalArgumentException(s:"Username can only contain letters, numbers, and underscores");
88     }
89 }
90
91 Tabnine | Edit | Test | Explain | Document
92 /**
93  * Validates password strength
94  * @param password the password to validate
95  * @throws IllegalArgumentException if password is invalid
96  */
97 public static void validatePassword(String password) throws IllegalArgumentException {
98     if (password == null || password.isEmpty()) {
99         throw new IllegalArgumentException(s:"Password cannot be empty");
100     }
101
102     if (password.length() < 6) {
103         throw new IllegalArgumentException(s:"Password must be at least 6 characters long");
104     }
105
106     if (password.length() > 50) {
107         throw new IllegalArgumentException(s:"Password cannot exceed 50 characters");
108     }
109 }
110
111 Tabnine | Edit | Test | Explain | Document
112 /**
113  * Validates name format
114  * @param name the name to validate
115  * @throws IllegalArgumentException if name is invalid
116  */
117 public static void validateName(String name) throws IllegalArgumentException {
118     if (name == null || name.trim().isEmpty()) {
119         throw new IllegalArgumentException(s:"Name cannot be empty");
120     }
121
122     if (name.trim().length() < 2) {
123         throw new IllegalArgumentException(s:"Name must be at least 2 characters long");
124     }
125
126     if (name.trim().length() > 50) {
127         throw new IllegalArgumentException(s:"Name cannot exceed 50 characters");
128     }
129
130     if (!name.matches(regex:"^[a-zA-Z\\s]+$")) {
131         throw new IllegalArgumentException(s:"Name can only contain letters and spaces");
132     }
133 }

```